BUSINESS REQUIREMENTS DOCUMENT

Customer Account & Service Management System

Internal Training Project — Software Engineering Internship

Document Version: 1.0
Date: August 2, 2026
Prepared By: Engineering Team
Audience: Intern Developer, Mentor / Reviewer
Technology Stack: .NET (Web API, EF Core) / Angular
Status: Draft for Review

1. Purpose

This document defines the business requirements for a Customer Account & Service Management System — a training project designed to give a new engineering team member hands-on experience with patterns commonly found in real banking systems. The system combines two connected domains: core account operations (customers, accounts, deposits, withdrawals, transaction history) and a customer service case-management layer (service requests, agent assignment, resolution workflow, audit notes).

The system is built for internal training purposes only and does not process real customer data or real funds.

2. Business Objectives

- Provide a safe, non-production environment to practice core banking account operations with strict balance and data integrity.
- Simulate a realistic customer service workflow, similar to how bank branches and call centers track and resolve customer requests.
- Introduce common banking system design patterns: transactional integrity, state machines, role-based access, and audit trails.
- Produce a working, demoable system that can be extended in future training phases (transfers, limits, interest, multi-branch support).

3. Scope

3.1 In Scope

- Customer management (create, view, update status)
- Account management (create, view, freeze/close status)
- Deposit and withdrawal transactions with balance validation
- Transaction history with pagination
- Service request lifecycle: creation, assignment, status transitions, resolution
- Automatic side effects of resolved requests (e.g. Account Freeze updates Account status)
- Audit trail via request notes
- Basic role-based views: Service Agent vs Supervisor

3.2 Out of Scope

- Real payment gateways or core banking system integration
- Interest calculation, loans, fixed deposits
- Multi-currency and multi-branch reconciliation
- Regulatory / compliance reporting
- Real customer data or production authentication infrastructure

4. Stakeholders

ID	Requirement
Mentor / Reviewer	Owns the requirements, reviews the intern's implementation and provides feedback.
Intern Developer	Designs and implements the backend (.NET) and frontend (Angular) based on this document.
Service Agent (simulated)	End user who manages assigned service requests.
Supervisor (simulated)	End user who oversees all requests and can reassign work.
Customer (simulated)	Represented via seed data; the party whose accounts and requests are managed in the system.

5. System Overview

The system is composed of six core entities: Customer, Account, Transaction, ServiceAgent, ServiceRequest, and RequestNote. A Customer owns one or more Accounts. Every deposit or withdrawal on an Account produces a Transaction record, which is the single source of truth for the Account balance. Separately, a Customer can raise a ServiceRequest (complaint, account freeze, information update, card issue, etc.), which is assigned to a ServiceAgent and moves through a defined status lifecycle. Every action taken on a ServiceRequest is logged as a RequestNote, forming a complete audit trail. Some request types have a direct side effect on banking data — for example, resolving an Account Freeze request changes the linked Account's status.

6. Business Rules

6.1 Account & Transactions

- An Account balance is never edited directly — it is always the result of applying Transactions.
- A withdrawal cannot exceed the current Account balance; the system must reject it with a clear error and create no Transaction record.
- Deposit and withdrawal amounts must be strictly positive; zero or negative amounts are rejected.
- Each balance-changing operation (validation, Transaction insert, balance update) must be atomic — wrapped in a single database transaction so concurrent operations cannot corrupt the balance.
- A Transaction record always stores the resulting BalanceAfter, so history is self-verifying.

6.2 Service Requests

- A ServiceRequest must always be linked to a Customer, and may optionally be linked to a specific Account.
- A ServiceRequest follows a defined state machine: New → InProgress → Resolved, or New → InProgress → Escalated → Resolved. Status cannot skip stages.
- Only the assigned Agent, or a Supervisor, may change a ServiceRequest's status or reassign it.
- Resolving an Account Freeze request must update the linked Account's status to Frozen as part of the same operation.
- Every status change or action on a ServiceRequest must create a corresponding RequestNote recording who did what and when.
- A Supervisor can view and reassign all requests; a Service Agent can view only requests assigned to them.

7. Functional Requirements

7.1 Customer Management

ID	Requirement
FR-1.1	The system shall allow creating a Customer record with Name, National ID, Email, and Phone.
FR-1.2	The system shall allow viewing a Customer's profile, including linked Accounts and ServiceRequests.
FR-1.3	The system shall allow updating a Customer's Status (Active / Blocked).

7.2 Account Management

ID	Requirement
FR-2.1	The system shall allow creating an Account linked to a Customer, with a unique Account Number.
FR-2.2	The system shall expose the current Account balance and status (Active / Frozen / Closed).
FR-2.3	The system shall prevent transactions on a Frozen or Closed Account.

7.3 Transaction Management

ID	Requirement
FR-3.1	The system shall provide a Deposit operation that increases the Account balance and records a Transaction.
FR-3.2	The system shall provide a Withdraw operation that validates sufficient funds before decreasing the balance.
FR-3.3	The system shall provide a paginated, most-recent-first Transaction history per Account.

7.4 Service Request Management

ID	Requirement
FR-4.1	The system shall allow creating a ServiceRequest with Type, Description, Priority, and an optional linked Account.
FR-4.2	The system shall allow assigning or reassigning a ServiceRequest to a ServiceAgent.
FR-4.3	The system shall enforce valid status transitions only, per the defined state machine.
FR-4.4	The system shall apply the correct side effect to related banking data when a request is resolved (e.g. Account Freeze).
FR-4.5	The system shall record a RequestNote for every action taken on a ServiceRequest.
FR-4.6	The system shall provide filtering of ServiceRequests by Status, Priority, and Assigned Agent.

7.5 Roles & Access

ID	Requirement
FR-5.1	The system shall restrict a Service Agent's request list to requests assigned to them.
FR-5.2	The system shall allow a Supervisor to view and reassign all ServiceRequests.
FR-5.3	The system shall present a dashboard summary of request counts by status.

8. User Stories

User stories are grouped by module. Each story includes acceptance criteria to guide implementation and review.

8.1 Module A — Accounts & Transactions

ID	As a	I want to	So that	Acceptance Criteria
US-01	customer	deposit money into my account	my balance increases correctly and is traceable	• Amount must be positive • A Transaction record is created • Balance is updated atomically with the Transaction insert
US-02	customer	withdraw money from my account	I can access my funds, as long as I have sufficient balance	• Withdrawal is rejected if amount exceeds balance • A clear error message is returned • No Transaction record is created on failure
US-03	customer	view my transaction history	I can track all activity on my account	• History is paginated • Sorted by date, most recent first • Each row shows type, amount, and balance after
US-04	system	guarantee balance integrity under concurrent requests	the account data can always be trusted	• Balance never goes negative • Validation, insert, and update happen in a single DB transaction • Concurrent deposit/withdraw requests do not corrupt the balance

8.2 Module B — Service Requests

ID	As a	I want to	So that	Acceptance Criteria
US-05	customer	submit a service request (complaint, freeze, info update, card issue)	my issue is tracked and resolved	• Request requires a Customer and a Type • Account link is optional • New request starts in status New
US-06	service agent	view the requests assigned to me	I can manage my daily workload	• List is filtered to the logged-in agent • Supports filtering by status and priority
US-	service agent	update a request's status	progress is tracked	• Only valid transitions are

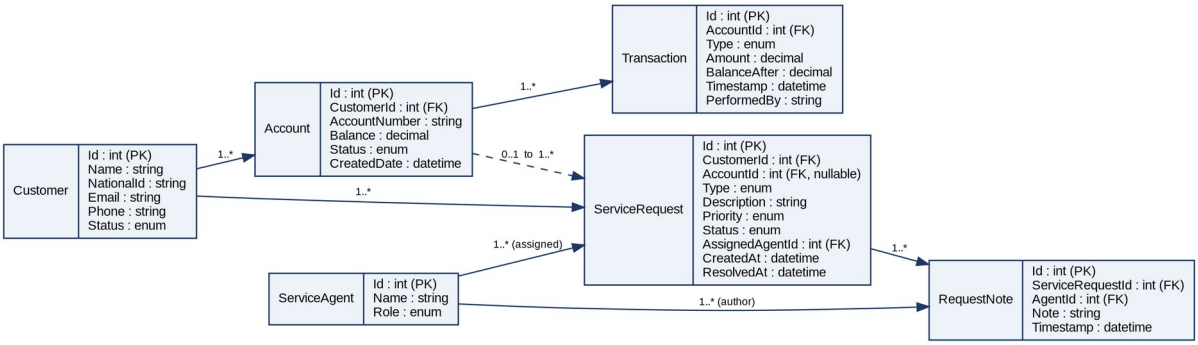
ID	As a	I want to	So that	Acceptance Criteria
07		following the defined lifecycle	accurately	allowed (e.g. cannot jump New → Resolved directly) • Invalid transitions return a clear error
US-08	service agent	have every action on a request logged automatically	there is a complete audit trail	• A RequestNote is created on every status change • Note stores agent, timestamp, and action
US-09	service agent	have an Account Freeze request automatically freeze the linked account when resolved	banking data reflects the resolution without a manual step	• Resolving an Account Freeze request updates Account.Status to Frozen • Update happens in the same operation as the resolution
US-10	supervisor	view all service requests, not just my own	I can monitor overall team performance	• Supervisor role sees requests from all agents • Same filtering options as the agent view
US-11	supervisor	reassign a request to a different agent	workload can be balanced across the team	• Only a Supervisor can reassign • Reassignment is logged as a RequestNote

8.3 Module C — Access & Dashboard

ID	As a	I want to	So that	Acceptance Criteria
US-12	system	restrict status updates to the assigned agent or a supervisor	unauthorized changes are prevented	• Non-assigned agents cannot update a request • Attempted unauthorized update returns 403 Forbidden
US-13	service agent	see a count of requests by status on a dashboard	I get a quick overview of my workload	• Dashboard shows counts for New / InProgress / Escalated / Resolved • Counts reflect only the agent's own requests (Supervisor sees all)

9. Entity Relationship Diagram

The diagram below shows the core entities, their key attributes, and the relationships between the account domain and the service request domain.



Note: Account → ServiceRequest is an optional relationship (0..1) since a request is not always linked to a specific account (e.g. a general complaint).

10. Assumptions & Constraints

- Single currency; no foreign exchange handling.
- No integration with external/core banking systems — this is a standalone simulation.
- Customers, accounts, and agents are provided via seed data rather than a live onboarding process.
- Authentication and role assignment can be simplified for training purposes (e.g. a selected user/role at login) rather than a full production identity system.
- No interest calculation, loan processing, or multi-branch reconciliation is included in this phase.

11. Non-Functional Notes

- Data integrity takes priority over performance optimization for this training scope.
- Code should follow the team's existing conventions for validation, error handling, and naming.
- The system should be structured so that future phases (transfers, limits, interest) can be added without a redesign.